Experiment 9

StudentName: Abhishek Sharma **UID: 23BCS12330**
Branch: BE CSE **Section/Group: KRG 3A**
Semester: 6th **Date of Performance: 20/04/26**
Subject Name: Full Stack Development **Subject Code: 23CSH-309**

Aim:
To design and implement a secure, scalable full-stack application using Spring Boot by integrating Spring Security, OAuth, Role-Based Access Control (RBAC), and JPA-based database performance optimization.

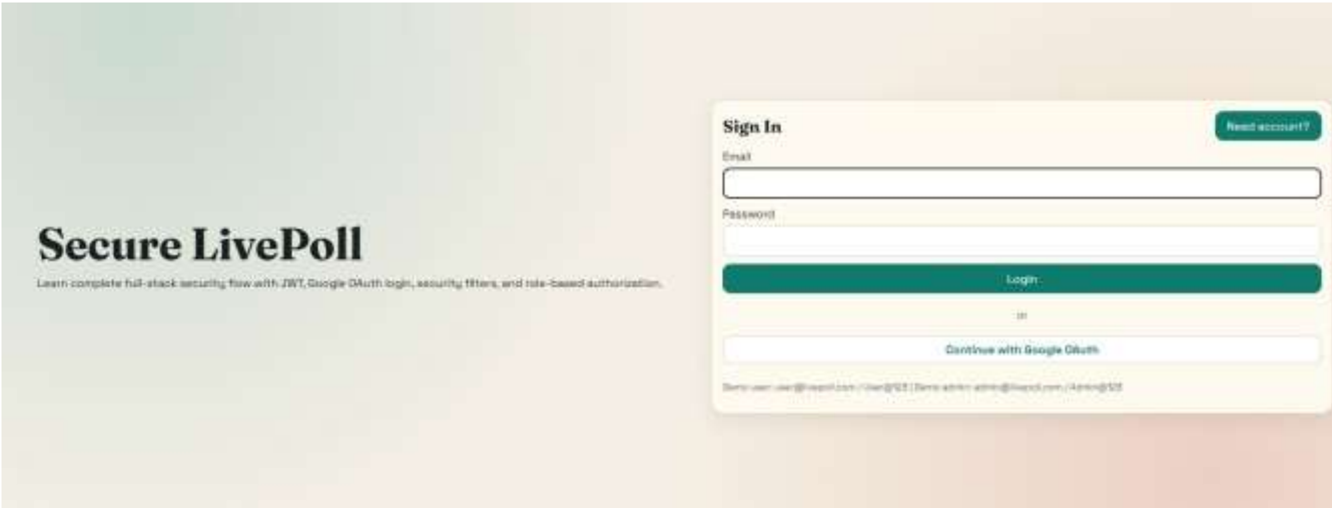
Objective:
1. Implement backend security using Spring Security and filter chains.
2. Enable authentication using OAuth (Google Login).
3. Apply Role-Based Access Control (RBAC) for endpoint protection.
4. Optimize database interactions using JPA and performance techniques.
5. Integrate a secure backend with a React frontend using CORS.

Implementation/Code:

AdminPollController.java
```java
package com.ecotrack.livepoll.controller;

import com.ecotrack.livepoll.auth.AppUserPrincipal;
import com.ecotrack.livepoll.dto.PollDtos;
import com.ecotrack.livepoll.service.PollService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
```

```java
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/admin/polls")
@PreAuthorize("hasRole('ADMIN')")
public class AdminPollController {

    private final PollService pollService;

    public AdminPollController(PollService pollService) {
        this.pollService = pollService;
    }

    @PostMapping
    public ResponseEntity<PollDtos.PollResponse> createPoll(@Valid @RequestBody
PollDtos.CreatePollRequest request,
                                                            @AuthenticationPrincipal AppUserPrincipal
principal) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(pollService.createPoll(request, principal.getUsername()));
    }

    @PostMapping("/{id}/close")
    public ResponseEntity<PollDtos.PollResponse> closePoll(@PathVariable Long id) {
        return ResponseEntity.ok(pollService.closePoll(id));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletePoll(@PathVariable Long id) {
        pollService.deletePoll(id);
        return ResponseEntity.noContent().build();
    }
}
```

Output:



Learning Outcome:

- Learned how Spring Security secures backend APIs
- Understood authentication vs authorization clearly
- Implemented OAuth 2.0 login using Google
- Gained experience with role-based access control (RBAC)
- Learned how to integrate React frontend with secured backend
- Understood complete request flow with security filters
- Improved debugging and analysis of secured applications